EMBERKIDS / PROJECT DELIVERY DOCUMENT

Software Requirements Specification
& Project Handover Record

A complete scope, acceptance, evidence and commercial handover record for the EmberKids Chess Academy platform.

Field	Record
Project	EmberKids Chess Academy Platform
Document owner	[Developer Name]
Client / academy	[Client Legal Name] / EmberKids Chess Academy
Version / date	1.0 • 21 July 2026
Status	Draft for scope confirmation and handover
Confidentiality	Confidential — commercial and technical record

Before sending

Replace every bracketed placeholder, insert the agreed amount and payment dates, attach the final repository/archive hash, and have a lawyer review the ownership and payment clauses. This document records the project; it is not legal advice.

1. Purpose, scope and evidentiary use

This document defines the implemented scope of the EmberKids Chess Academy website and academy-management platform, records the current handover expectations, and provides a dated evidence register. It is intended to be signed by the developer and client so both parties have a shared record of what was delivered, what remains client-owned or client-supplied, and how payment and maintenance are handled.

The document is written as a project record, not as a replacement for a lawyer-drafted development agreement. Where a clause affects ownership, licensing, payment recovery or enforceability, the parties should obtain legal review under the applicable Indian law and identify the correct legal entity and signatory.

1.1 Document conventions

- Implemented / in scope means the capability is represented in the current codebase and should be verified jointly during acceptance.
- Existing-feature bug means a reproducible defect in a feature listed in this SRS, using supported environments, without a new requirement or unauthorized code change.
- Client-provided means the client supplies the account, credentials, content, approvals, hosting, domain or third-party subscription needed for production operation.
- Bracketed values such as [Final Amount] are fill-in fields and must be completed before signature.

2. Product overview

EmberKids is a responsive chess-academy platform combining a public marketing site, secure admin operations, staff/coach workflows, student self-service, enrolment and payment operations, class and attendance tracking, evaluation/report cards, notifications, and an academy-focused AI guide named Amber.

2.1 Solution architecture recorded in the codebase

Layer	Recorded implementation
Frontend	Next.js / React / TypeScript responsive web application with public, admin, staff and student routes.
Backend	Express / Node.js REST API with controllers, services, validation, scheduled jobs and integrations.
Data	MongoDB models for academy, learner, staff, course, batch, attendance, payments, notifications, audit and knowledge data.
Infrastructure	Redis-compatible caching is supported/recommended; production requires secure environment configuration, SMTP and a MongoDB replica set for transactions.
Integrations	Email/SMTP, WhatsApp messaging, Cloudinary/file storage, payment links/Wise flows and Gemini Flash + embeddings for the academy chatbot.

Configuration note

The AI model is selected server-side through configuration. The original project scope called for Gemini 2.5 Flash; the deployed model name should be confirmed from the production environment before final acceptance. Never place an API key, password or private token in this SRS.

3. Implemented feature inventory

The following inventory is grouped for the client call and for acceptance testing. Each group should be checked against the live deployment before the final sign-off.

Area	Included capability	Status
Public site	Home/landing page, academy positioning, calls-to-action, about, courses and roadmap, coaches, prodigies, testimonials, FAQ, contact, privacy and terms pages; responsive layouts, theme styling, animation, forms and contact links.	In scope
Admissions and leads	Free-trial/admission enquiry capture, lead records, follow-up workflow, contact details, demo-class journey and lead status management.	In scope
Admin portal	Secure admin login/password reset, dashboard, role/permission controls, students, leads, courses/roadmap, testimonials, site content, coaches/staff, batches/history, classes, trial classes, events/masterclasses, tournaments, packages, payment links, payments, exports, reports and audit logs.	In scope
Staff / coach portal	Staff login, dashboard, assigned classes and batches, student lists, attendance and disputes, trial classes, report cards/evaluations, reports and payment-link/salary-related workflows represented by the current modules.	In scope
Student portal	Student login/password reset, dashboard, upcoming classes, schedule and batch information, attendance, evaluation/report cards, packages/payments, notifications, WhatsApp group link, profile and account flows.	In scope
Core API and security	REST routes, controllers/services, MongoDB models, refresh-token/session handling, role-based permissions, secure cookies, CORS/Helmet/sanitisation, rate limits, audit logs, transactions, scheduled jobs, email, WhatsApp, uploads/storage and CSV exports.	In scope
Amber AI guide	Gemini Flash chat integration, website/content knowledge chunks, embeddings/indexing, retrieval-grounded prompts, academy-only boundary, out-of-scope fallback with consultant contact, fee enquiries routed to the academy team, clear batch-format explanation, streaming responses, session history, typing/loading/error states, responsive popup, Amber mascot launcher and greeting/tap audio.	In scope

3.1 Academy-specific chatbot behavior

- Amber answers academy-related questions using retrieved academy knowledge first, including courses, batches, fees, FAQs, instructors, policies and contact details available in the indexed content.
- For unrelated questions, Amber politely states that it can assist only with EmberKids academy information and offers the academy contact route.
- For fee questions, Amber directs the visitor to the academy consultant/team for the current fee conversation and provides configured contact details rather than inventing a price.
- For batch-size questions, Amber explains the three formats clearly: 1:1, Premium Group (2–3 students), and Standard Group (5–6 students), subject to the academy’s current availability.

- The launcher is designed to sit near the WhatsApp action without overlap; the tooltip and message behavior are part of the UI acceptance check.

4. Functional requirements and acceptance matrix

ID	Requirement	Acceptance criterion	Verification
FR-01	Public navigation	All public routes load, navigation and responsive layout work, and key CTA/contact links resolve.	Joint UAT
FR-02	Enquiry / trial capture	A visitor can submit a supported enquiry or trial request; validation and lead creation are visible to authorised staff/admin.	Joint UAT
FR-03	Admin operations	Authorised admin can manage the listed academy records, content, batches, schedules, payments and reports.	Joint UAT
FR-04	Staff workflows	Authorised staff can view assigned work, attendance, learners, trial classes and evaluation/report modules allowed by permissions.	Joint UAT
FR-05	Student self-service	A student can authenticate and view their schedule, attendance, report cards, payments/packages and notifications as configured.	Joint UAT
FR-06	Payments and links	Configured payment links and status callbacks create the expected payment/enrolment state; provider availability is a dependency.	Joint UAT
FR-07	Notifications	Configured email/WhatsApp/in-app notifications are generated for supported events and failures are observable in logs/audit records.	Joint UAT
FR-08	Amber RAG chat	Every query retrieves relevant knowledge before generation, streaming works when provider is available, and out-of-scope/fee/batch rules are respected.	Joint UAT
FR-09	Chat UX	Launcher, popup, typing indicator, auto-scroll, Enter/Shift+Enter, retry/error state, current-session history, responsive width/height and no-overlap placement work on supported breakpoints.	Joint UAT
FR-10	Audit and export	Authorised users can access supported audit records and CSV/report exports without exposing secrets.	Joint UAT

5. Non-functional requirements

Quality area	Requirement
Responsive UX	Public, portal and chatbot layouts adapt to desktop, tablet and small screens without overlap or unusable controls.
Performance	Keep client bundles, network requests, database queries and RAG context focused on the current task; exact production targets depend on hosting and third-party latency.
Security	Use server-side secrets, secure cookies/tokens, access control, validation/sanitisation, rate limiting, security headers, least privilege and audit trails.
Availability	Third-party services (Gemini, SMTP, WhatsApp, payment providers, storage and hosting) may be unavailable; the UI must show graceful error/retry states.
Accessibility	Use keyboard-accessible controls, visible focus, labels, sensible contrast and reduced-motion-friendly behavior where supported.
Privacy	Do not store or publish secrets in source, screenshots, SRS or logs. Collect only the learner/contact data needed for academy operations and apply the client's privacy policy.
Maintainability	Keep environment variables documented, modules separated by responsibility, migrations/backups planned, and changes recorded in Git and the change log.

6. Assumptions, dependencies and exclusions

6.1 Client responsibilities and dependencies

- The client supplies and pays for hosting, domain, DNS/SSL, database/Redis services, SMTP, Gemini/API, WhatsApp, Cloudinary/storage, payment-provider and other third-party accounts or usage charges.
- The client supplies accurate academy content, pricing/fee approvals, batch availability, contact numbers, policies, logos/assets, legal text and production credentials in a secure channel.
- The client approves UAT findings, signs the acceptance record, maintains backups and controls production account access after handover.
- Provider/model changes, quotas, rate limits, account suspension, network outages or changes made by third parties are dependencies and may require a separate change request.

6.2 Exclusions unless separately agreed

- New features, new portal roles, new pages, new integrations, major redesigns, rebranding, new reports or changes to business rules after acceptance.
- Bulk content entry, data cleansing/migration, ongoing academy operations, learner support, 24/7 monitoring, hosting/domain fees, third-party subscriptions and provider usage fees.
- Security certification, legal/compliance drafting, penetration testing, SEO/marketing campaigns or guarantees of third-party ranking/deliverability.
- Fixes caused by unauthorised code changes, unsupported environments, client infrastructure misconfiguration, expired credentials or third-party outages.

7. Handover, payment and intellectual-property record

Important

A repository or hosting account being created in the client's name does not, by itself, prove that source-code ownership or payment was transferred. The parties should sign this record (or a separate development agreement), keep the payment trail, and create a dated source archive/hash before final handover.

7.1 Proposed handover package

#	Item	Record
1	Signed SRS and acceptance record	Scope, exclusions, acceptance notes and signatures.
2	Source repository / archive	Source code, Git history, build files and a dated SHA-256 hash.
3	Deployment pack	README, environment-variable template, build/start instructions and rollback notes; no secret values.
4	Data and integration notes	Models/routes/integrations, provider assumptions, backup expectations and account ownership list.
5	Evidence pack	Dated screenshots, commit log, change log and test/UAT notes.

7.2 Payment and rights clause to complete and sign

The commercial fields below are intentionally blank until the parties enter the agreed values:

Commercial field	Value	Notes
Agreed project amount	[₹_____]	Final amount including/excluding taxes: [_____]
Advance / milestone 1	[₹_____] due [date]	Trigger: [_____]
Milestone 2	[₹_____] due [date]	Trigger: [_____]
Final payment	[₹_____] due [date]	Due before full source/IP handover
Outstanding at signature	[₹_____]	To be acknowledged by both parties

1. Until the agreed amount is paid in full and cleared, the developer retains ownership of the source code and deliverables, and the client receives only the limited evaluation/use permission expressly agreed in writing.
2. After full cleared payment and written handover, the developer transfers or licenses the agreed rights to the client as stated in the signed commercial agreement. The exact legal form of the transfer should be confirmed by counsel.
3. Before full payment and written handover, the client must not copy, resell, sublicense, commercialise, publish, modify for another provider, or distribute the source code except as expressly agreed in writing.
4. No API key, password, private token, database credential or personal data is part of the SRS. Credentials must be exchanged separately and rotated at handover.
5. If the client requests work beyond the listed scope, the parties should approve a written change request with effort, price and delivery effect before that work begins.

8. Three-month maintenance policy

Promised support window

The developer is providing three (3) months of maintenance after [written acceptance / production launch — choose one], subject to the limits below. This is a limited bug-fix window for existing implemented features, not an open-ended development retainer.

Policy item	Definition
Included	Reproducible bugs in features listed in this SRS that were already implemented and accepted, when used with the agreed production configuration and supported browsers/devices.
Included	Reasonable diagnosis, a corrective code change, and a verification/deployment note for the existing feature where the developer still has authorised access.
Response	Client reports should include page/feature, steps to reproduce, expected result, actual result, timestamp and screenshots/video where useful. The developer will acknowledge and prioritise reasonably; no 24/7 SLA is promised unless separately signed.
Not included	New features, change requests, redesigns, content changes, new integrations, new reports, new roles, data entry/migration or changes to business rules.
Not included	Hosting/domain/DNS/SSL, expired credentials, third-party provider outages or API/model/quota

Policy item	Definition
	changes, payment/WhatsApp/SMTP/storage issues outside the developer's code, or client-side misconfiguration.
Not included	Defects caused by unauthorised edits, another vendor, unsupported browser/device, malicious activity, force majeure or a production environment that differs materially from the accepted configuration.
After window	Requests after the three-month window, or excluded work during the window, require a separate quote or maintenance agreement.

Maintenance start date: [_____] • Maintenance end date: [_____] • Support channel: [_____]

9. Acceptance, sign-off and change control

The parties should complete a joint UAT review of the listed acceptance criteria. The client should provide any defect list in writing with reproduction details during the agreed review period. New requests or preference changes are not defects and should be logged as change requests.

Acceptance field	Value
Review period	[__] calendar days from delivery / access
Accepted build / URL	[Production URL or release tag]
Open defects	[None / list ticket IDs in attachment]
Acceptance date	[DD/MM/YYYY]
Payment due on acceptance	[₹_____] due [date]

Party	Name	Signature	Date
Developer / project author	[Name]	Signature: _____	Date: _____
Client / authorised signatory	[Legal name + role]	Signature: _____	Date: _____

Appendix A — Dated UI evidence

The following screenshots were captured from the local preview on 21 July 2026. They show the visible website implementation only; confirm the hosted build and current data before signing. No secrets are included.

Figure A-1. Public homepage viewport — brand, navigation, hero CTA and floating actions.

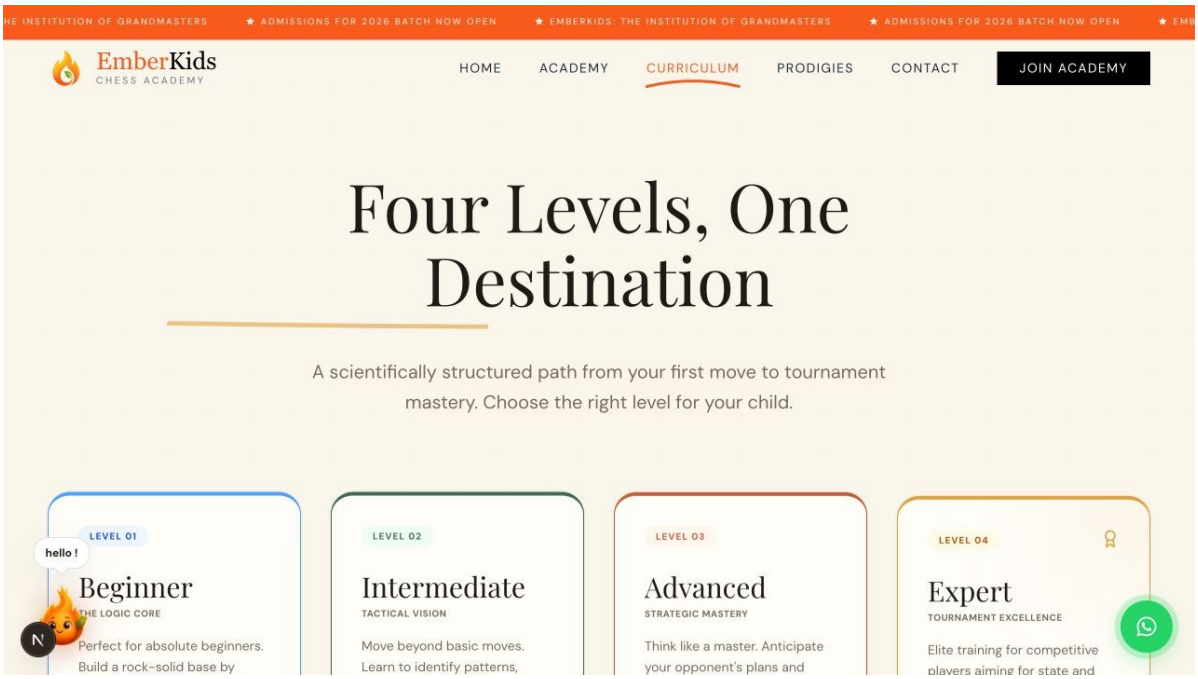

Figure A-2. Public courses page viewport — course/roadmap presentation and responsive site styling.

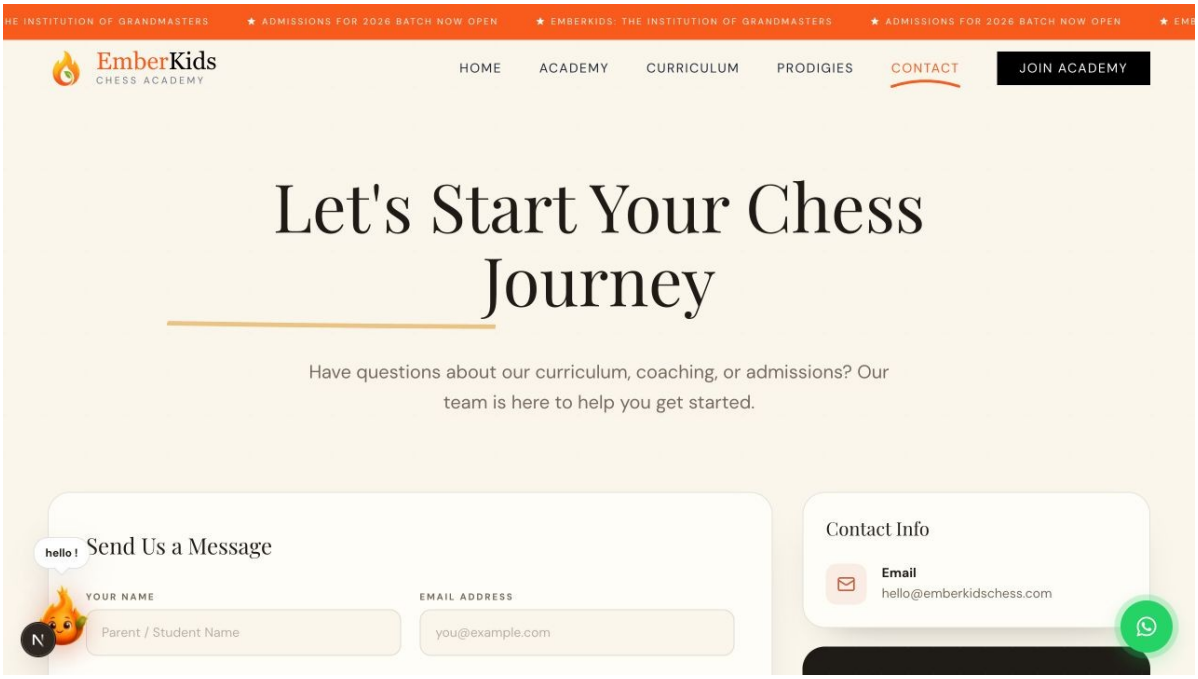

Figure A-3. Public contact page viewport — enquiry/contact route and academy-facing information.

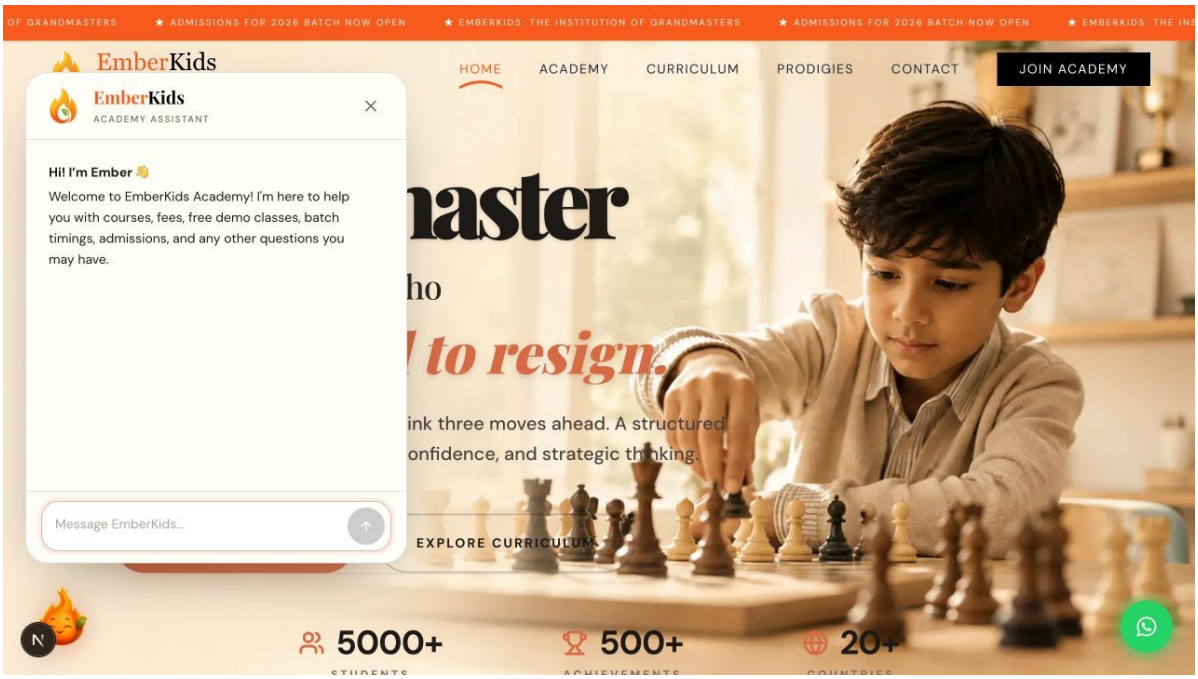

Figure A-4. Amber chatbot popup — launcher interaction, academy greeting, chat input and responsive visual treatment.

Appendix B — Evidence register and authorship record

ID	File	What it records	Source	Captured
A-1	homepage.png	Public homepage and floating actions	Local preview	21 Jul 2026
A-2	courses.png	Courses/roadmap presentation	Local preview	21 Jul 2026
A-3	contact.png	Academy contact route	Local preview	21 Jul 2026
A-4	chatbot-popup.png	Amber chatbot popup and launcher	Local preview	21 Jul 2026

B.1 Repository snapshot to attach before handover

Evidence item	Value	Action / interpretation
Current branch at evidence review	main	Create a signed release tag for final handover.
Recent commit	e67878a	Update chatbot components and add new audio file (19 Jul 2026).
Previous chatbot commit	88820b4	Add chatbot feature with knowledge base and UI components (19 Jul 2026).
Tracked code footprint	42,863 lines	TS/TSX/JS/JSX/CSS in frontend and backend at evidence review.
Module counts	51 components • 14 services • 28 models • 29 routes	Use Git export to preserve the exact final snapshot.
Final archive hash	[SHA-256: _____]	Compute after final code freeze; store with this signed record.

Authorship protection checklist

Before handover: commit all final changes, create a signed Git tag, export `git log` and `git status`, create a source archive, compute SHA-256, email this SRS and the hash from the developer's own account, obtain written client acknowledgement, and keep the payment invoices/receipts. These steps create a stronger contemporaneous record than a screenshot alone.

Appendix C — Change log

Version	Date	Author	Change
1.0	21 Jul 2026	[Developer Name]	Initial SRS, feature inventory, acceptance matrix, handover/payment clauses, 3-month maintenance policy and dated UI evidence.
[ ]	[date]	[name]	[Future approved change]

Appendix D — Final handover acknowledgement

By signing below, the parties confirm that they have reviewed this SRS, understand the listed scope and exclusions, and will use the agreed commercial agreement/payment record to determine when source-code/IP rights and full handover become effective.

Party	Acknowledgement	Signature
Client acknowledgement	I confirm receipt of the SRS and the listed scope/evidence. Outstanding payment and acceptance fields are recorded above.	Name/signature/date: _____ _____
Developer acknowledgement	I confirm that the listed implementation and maintenance promise reflect the agreed project record, subject to final UAT and the signed commercial terms.	Name/signature/date: _____ _____

Final reminder

Do not place production API keys, passwords or private client data in this document. Complete the bracketed fields, sign every page or initial the change/evidence pages as appropriate, and obtain legal review for the payment/IP language before relying on it in a dispute.